

CEUB | Desenvolvimento de Sistemas

Roteiro Prático - Aula 10 | Clean Code, refatoração curta e Classroom 50

Produto esperado

- Repositório individual da Aula 10 criado pelo assignment no Classroom 50.
- Projeto Java importado no Eclipse como projeto existente.
- Código executado antes e depois da refatoração.
- AlunoService refatorado com constantes e método auxiliar para validar semestre.
- README atualizado com o que foi alterado e quais testes foram realizados.
- Commit e push feitos pelo PowerShell.

Regra da aula

Refatorar não é criar uma funcionalidade grande. A meta é melhorar a clareza do código mantendo o mesmo comportamento.

1. Preparar o repositório individual

1. Abra o link do assignment individual da Aula 10 no Classroom 50.
2. Aceite a atividade com sua própria conta GitHub.
3. Aguarde a criação do repositório individual.
4. No GitHub, clique em Code > HTTPS e copie a URL do repositório.
5. Abra o PowerShell em uma pasta simples, por exemplo C:\DesenvolvimentoSistemas.
6. Execute os comandos abaixo.

```
git --version
git clone URL_DO_REPOSITORIO
cd NOME_DO_REPOSITORIO
git status
dir -Force
```

Checkpoint 1

A pasta deve conter .project, .classpath, README.md e src. Se o Git não for reconhecido, pare e chame a professora.

2. Importar e executar no Eclipse

7. Abra o Eclipse no workspace da disciplina.
8. Acesse File > Import.
9. Escolha General > Existing Projects into Workspace.
10. Selecione a pasta clonada que contém diretamente .project e src.
11. Deixe Copy projects into workspace desmarcado.
12. Clique em Finish.

13. Abra Principal.java no pacote app.
14. Execute com Run As > Java Application.
15. Cadastre dois alunos e teste listar, buscar e listar por semestre.

Checkpoint 2

Não altere código antes de confirmar que Principal.java executa corretamente.

3. Conceito de Clean Code usado na prática

Nesta prática, vamos melhorar a legibilidade de uma validação que aparece em mais de um ponto do sistema. Em vez de deixar números soltos no código, vamos criar constantes. Em vez de repetir a condição `semestre < 1 || semestre > 10`, vamos criar um método auxiliar com nome claro.

4. Prática guiada - refatorar validação de semestre

Abra AlunoService.java. Localize os pontos em que aparece a regra de semestre entre 1 e 10. A regra pode aparecer em cadastrar e listarPorSemestre.

Passo 1 - Criar constantes no início da classe, logo abaixo do atributo repository:

```
private static final int SEMESTRE_MINIMO = 1;  
private static final int SEMESTRE_MAXIMO = 10;
```

Passo 2 - Criar o método auxiliar no final da classe, antes do último fechamento da chave:

```
private boolean semestreInvalido(int semestre) {  
    return semestre < SEMESTRE_MINIMO || semestre > SEMESTRE_MAXIMO;  
}
```

Passo 3 - Substituir as condições antigas por chamadas ao método auxiliar:

```
if (semestreInvalido(semestre)) {  
    return "Semestre deve estar entre " + SEMESTRE_MINIMO + " e " + SEMESTRE_MAXIMO + ".";  
}
```

Passo 4 - Salvar, executar Principal.java e testar: cadastrar semestre 0, cadastrar semestre 11, cadastrar semestre 3 e listar por semestre 3.

Checkpoint 3

O comportamento deve continuar igual para o usuário, mas o código fica mais legível para quem vai manter o sistema.

5. Atualizar README e entregar

- No README, preencha a seção Aula 10.
- Informe que a refatoração criou constantes para a faixa de semestre.
- Informe que a regra foi concentrada no método `semestreInvalido`.
- Registre os testes executados.

```
git status  
git add README.md src  
git commit -m "refatora validacao de semestre"  
git push  
git log --oneline
```

6. Atividade individual

Depois da prática guiada, faça a atividade impressa da Aula 10. A alteração será curta: melhorar a formatação da listagem de alunos sem criar nova funcionalidade. Use o mesmo repositório individual e entregue com commit e push.